

Economic System Simulator

Technical Documentation

Author: AiPixelCode

Version: 1.0

July 2026

Introduction

The Economic System Simulator is an object-oriented Python project that models a simplified economic environment consisting of people and workplaces.

The project demonstrates how inheritance, abstraction, polymorphism, composition, class variables, and custom exceptions can be combined to build a structured simulation where people work in different workplaces, generate income, pay living expenses, and contribute to the overall economy.

The implementation focuses on clean class design and interaction between independent objects rather than user interfaces or persistent data storage.

Project Objectives

The project was developed to demonstrate the practical application of object-oriented programming concepts, including:

- Class design
- Inheritance
- Method overriding
- Abstract behavior through base classes
- Object composition
- Static methods
- Class variables
- Custom exceptions
- Multi-class interaction

Project Structure

`economic_system_simulator/`

```
|— person.py
|— worker.py
|— teacher.py
|— engineer.py
|— work_place.py
|— mine.py
|— school.py
|— company.py
|— main.py
```

System Architecture

The simulator is divided into two major components.

Human Resources

- Person
- Worker
- Teacher
- Engineer

Workplaces

- WorkPlace
- Mine
- School
- Company

The two components communicate through the hiring process.

Each employee stores a reference to its workplace, while every workplace stores a list of its employees.

Class Descriptions

Person

The base class representing every individual in the system.

Attributes:

- name
- age
- level
- job
- work_place

Responsibilities:

- Store personal information
- Upgrade experience level
- Calculate net daily balance
- Keep track of every created person

Worker

Represents a worker.

Provides implementations for:

- Hiring price
- Daily income
- Living cost

Teacher

Represents a teacher.

Implements profession-specific formulas for:

- Hiring price
- Income
- Living cost

Engineer

Represents an engineer.

Calculates:

- Hiring price
- Daily income
- Living expenses

using formulas different from the other professions.

WorkPlace

Base class for all workplaces.

Attributes:

- name
- level
- expertise
- employees
- capacity

Responsibilities:

- Hire employees
- Upgrade workplace level
- Calculate maintenance costs
- Track every created workplace

Mine

Represents a mining workplace.

Provides customized implementations for:

- Capacity calculation
- Maintenance cost calculation

School

Represents an educational workplace.

Implements its own capacity and maintenance formulas.

Company

Represents a company workplace.

Provides company-specific calculations for:

- Capacity
- Daily costs

WorkPlaceIsFull

Custom exception raised whenever an attempt is made to hire an employee after reaching workplace capacity.

Financial Model

Each profession calculates:

- Hiring price
- Daily income
- Daily living cost

Net balance is calculated as:

$$\text{Net Balance} = (\text{Level-adjusted Income}) - \text{Living Cost}$$

Every workplace calculates:

- Construction price
- Daily maintenance cost

Its financial balance is represented as a negative daily expense.

Level System

Both people and workplaces can be upgraded.

Person Upgrade

Increasing a person's level improves the effective income through the level multiplier.

Workplace Upgrade

Increasing workplace level affects:

- Capacity
- Maintenance cost

depending on workplace type.

Capacity Management

Each workplace has a maximum employee capacity.

Hiring process:

1. Check current employee count.
2. Compare with capacity.
3. Hire employee if space exists.
4. Otherwise raise: `WorkPlaceIsFull`

Financial Aggregation

The project uses static methods to calculate global statistics.

Person

`Person.calc_all()`: Returns the combined daily financial balance of every person.

WorkPlace

`WorkPlace.calc_all()`: Returns the combined daily balance of all workplaces.

Overall economy:

$\text{Overall Balance} = \text{People Balance} + \text{Workplace Balance}$

Demonstration Program

The main.py file demonstrates all implemented features, including:

- Creating people
- Creating workplaces
- Workplace upgrades
- Hiring employees
- Capacity checking
- Employee upgrades
- Individual financial calculations
- Workplace financial calculations
- Overall economy summary
- Custom exception handling

The output is formatted into multiple sections to clearly present each stage of the simulation.

Object-Oriented Concepts Used

This project demonstrates the following OOP principles:

- Encapsulation
- Inheritance
- Method overriding
- Composition
- Polymorphism through shared interfaces
- Static methods
- Class variables
- Custom exceptions
- Separation of responsibilities

Conclusion

The Economic System Simulator illustrates how object-oriented programming can be used to model interactions between independent entities in a structured and maintainable way.

By separating responsibilities among specialized classes and defining clear relationships between people and workplaces, the project provides a solid foundation for experimenting with object-oriented software design while serving as a practical example of Python OOP design principles.